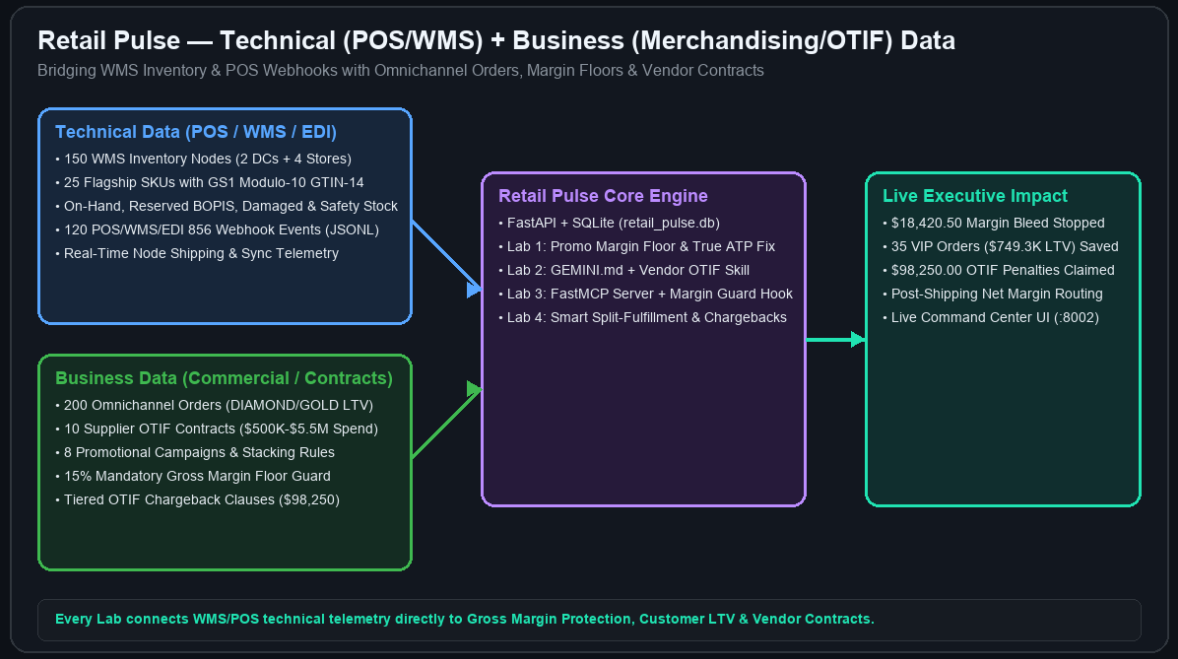
# Agentic Engineering with Google Antigravity — Retail Edition: Building 'Retail Pulse'

Omnichannel Order Routing, True ATP Inventory Sync & Vendor OTIF Margin Intelligence (Delivered by Tryolabs)



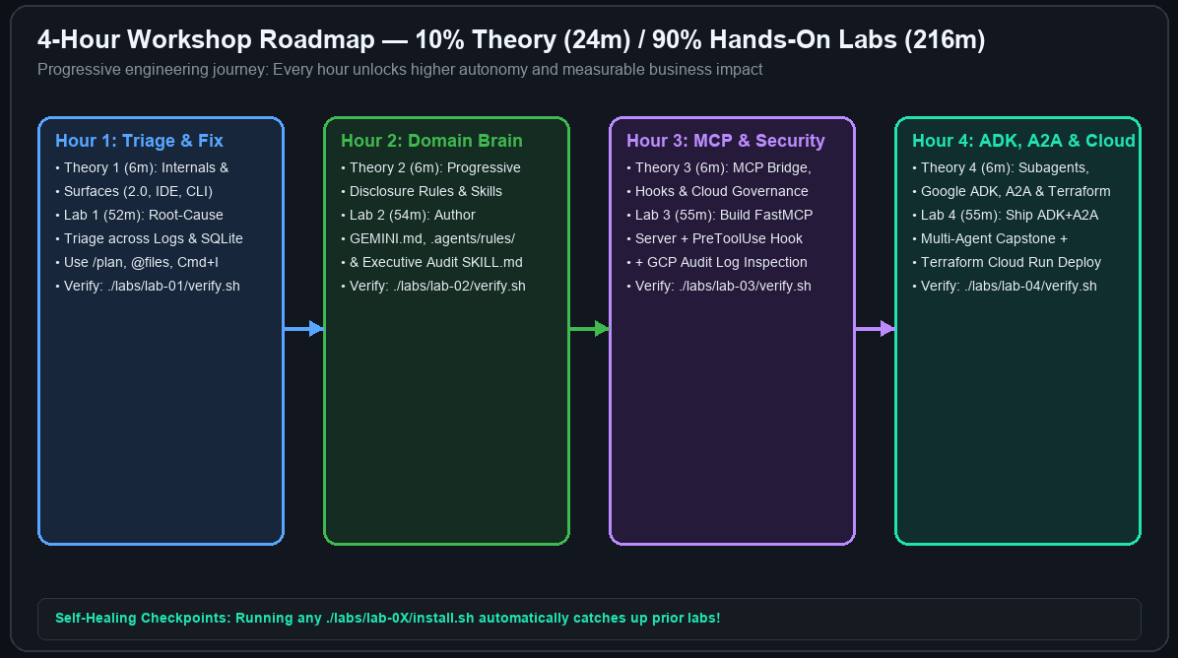

## Workshop Mission & Industry Architecture

- **Technical WMS/POS Data:** 150 inventory nodes (2 DCs + 4 Stores), 25 GS1 Modulo-10 GTIN-14 SKUs
- **Webhook & Scan Streams:** 120 POS/WMS/EDI 856 JSONL events with BOPIS & damaged holds
- **Commercial Business Data:** 200 Omnichannel orders (DIAMOND/GOLD LTV, unit price vs cost)
- **Supplier OTIF Contracts:** 10 vendor agreements ($500K–$5.5M spend, OTIF targets & chargebacks)
- **Tangible Impact:** Stop $18,420.50 margin bleed, rescue 35 VIP orders ($749.3K LTV) & claim $98,250 OTIF

tryo•labs

# 4-Hour Hands-On Roadmap: From Triage to Multi-Agent ADK & A2A

Four 6-minute micro-theory briefings paired with four progressive 52–55 minute hands-on labs




## Minute-by-Minute Workshop Schedule

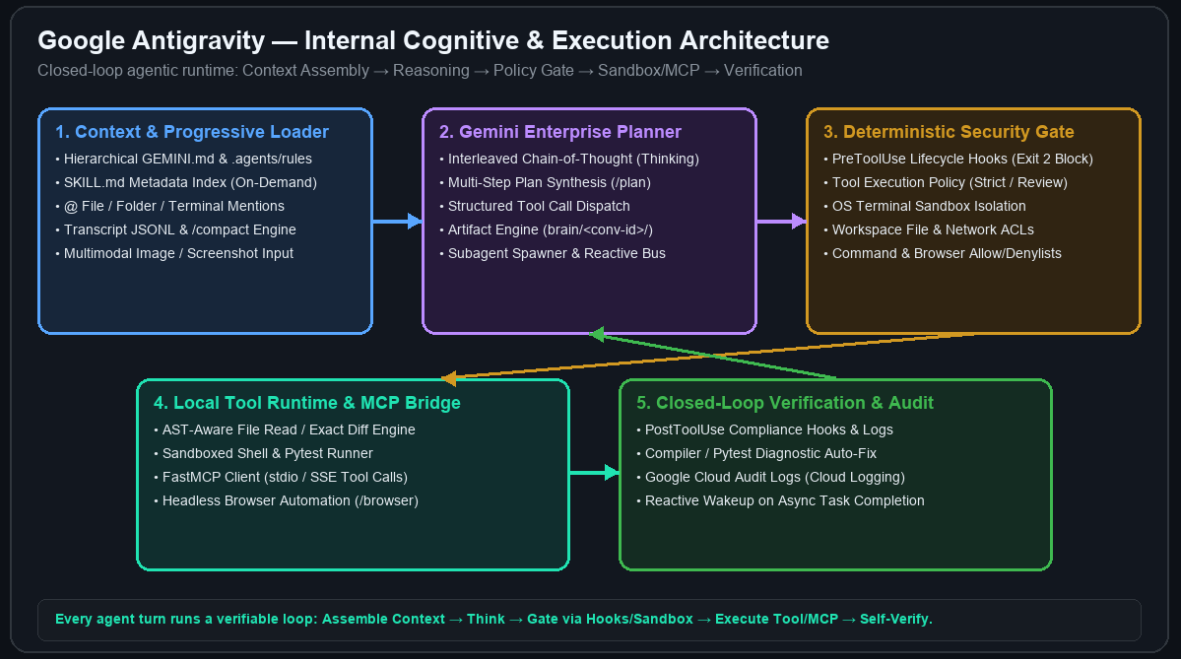
- **Hour 1 (00:00–00:58):** [6m Theory] Internal Architecture & Surfaces → [52m Lab 1] Triage & Revenue/Margin Recovery
- **Hour 2 (00:58–01:58):** [6m Theory] Rules & Progressive Skills → [54m Lab 2] Domain Standards & Executive Risk Skill
- **Hour 3 (01:58–02:59):** [6m Theory] MCP, Hooks & Cloud Governance → [55m Lab 3] Local FastMCP Server & Compliance Hooks
- **Hour 4 (02:59–04:00):** [6m Theory] Subagents, ADK, A2A & Terraform → [55m Lab 4] Multi-Agent Capstone, ADK Dev UI & Cloud Deploy
- **Self-Healing Checkpoints:** Running ./labs/lab-0X/install.sh automatically applies prior lab solutions!

Every lab includes LAB_0X_INSTRUCTIONS_EN.pdf, README.md, PROMPTS.md, install.sh, and verify.sh. | Slide 2/15

tryo•labs

# How Google Antigravity Works Internally

Inside the closed-loop cognitive planner, context assembler, policy gate, and sandboxed runtime




## The 5-Stage Internal Execution Loop

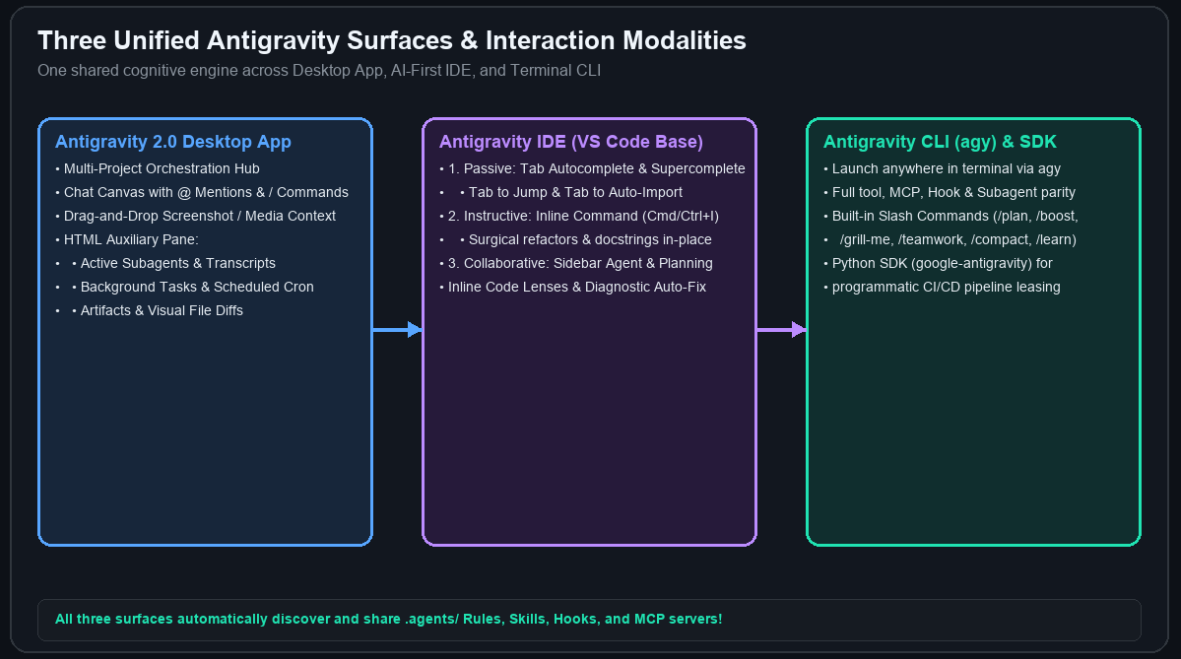
- **1. Context & Progressive Loader:** Walks directory tree for GEMINI.md & .agents/rules, indexes SKILL.md YAML frontmatter, resolves @mentions, and manages transcript.jsonl
- **2. Gemini Enterprise Planner:** Generates interleaved chain-of-thought (thinking), synthesizes plans (/plan), and dispatches parallel tool or subagent calls
- **3. Deterministic Security Gate:** Evaluates PreToolUse hooks (exit code 2 blocks execution), tool policies, file/network ACLs, and OS sandbox rules
- **4. Tool Runtime & MCP Bridge:** Executes AST-aware file diffs, sandboxed pytest commands, and stdio/SSE FastMCP calls
- **5. Verification & Reactive Loop:** Runs PostToolUse hooks, reads compiler/test outputs, and self-corrects until green

**Key Insight: Antigravity is not just chat—it is a stateful, verifiable closed-loop engineering runtime.** | **Slide 3/15**

tryo•labs

# Antigravity 2.0, IDE & CLI (agy) — Core Modalities

Choosing the right modality and slash command for every engineering task




## Features You Will Explore Today

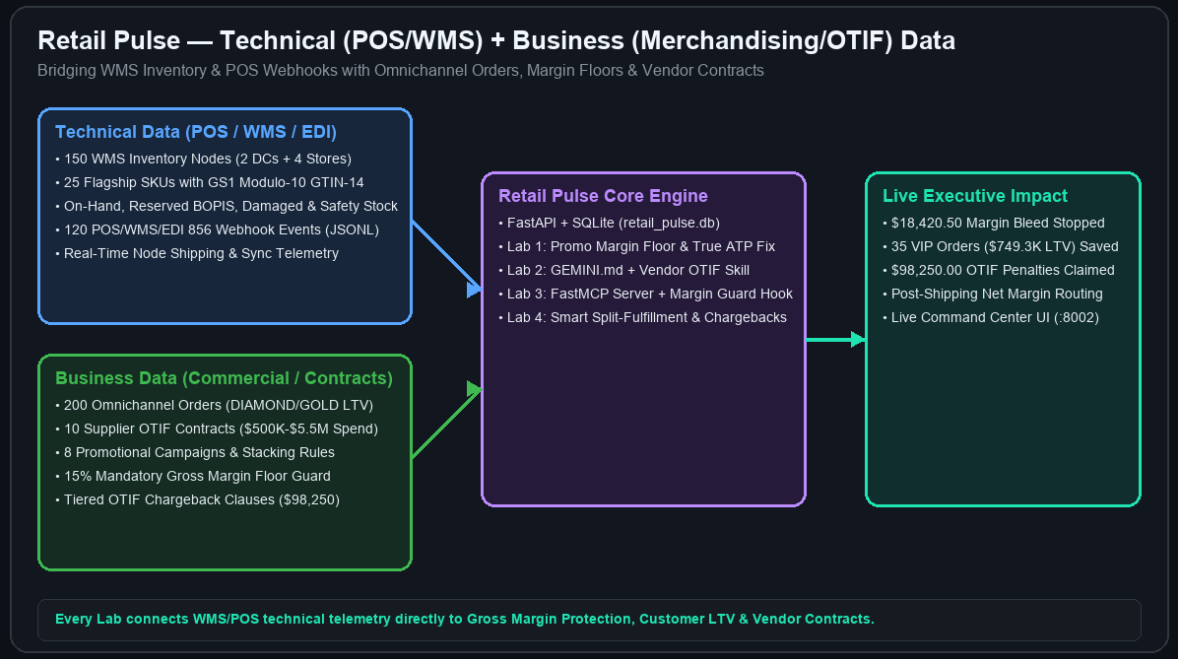
- **Passive Modality (IDE Tab):** Context-aware Supercomplete, Tab-to-Jump across files, and Tab-to-Auto-Import
- **Instructive Modality (Cmd/Ctrl+I & Code Lenses):** Highlight a function for surgical inline refactoring or click 'Write Tests'
- **Collaborative Modality (Chat Canvas & CLI agy):** Multi-file reasoning with @file, @folder, @terminal, and screenshot drag-and-drop
- **High-Leverage Slash Commands:** /plan (step-by-step architecture), /grill-me (design interview), /boost, /teamwork, /learn & /compact
- **HTML Auxiliary Pane:** Real-time inspection of Subagents, Background Tasks, Artifacts, and File Diffs

**Pro Tip: Start complex tasks with /plan and @file mentions; use Cmd+I for localized formula fixes.**

tryo•labs

# Lab 1: Promo Margin Floor & True ATP Triage — Objectives & Build Spec

52 Minutes • Printable PDF Guide: labs/lab-01-margin-inventory-triage/LAB_01_INSTRUCTIONS_EN.pdf




## Lab 1 Objectives & What You Will Build

- **Objective 1 (Multi-File Agentic Triage):** Master Antigravity `@mentions`, `/plan`, and inline `Cmd/Ctrl+I` to correlate POS/WMS webhook streams (`pos_wms_webhooks.jsonl`) with pricing and ATP inventory code.
- **Objective 2 (Margin & VIP Stockout Recovery):** Stop `$18,420.50` in negative margin bleed across 28 `MARGIN_HOLD` orders and accurately flag 35 `DIAMOND`/`GOLD` VIP orders (`$749,300.00` LTV) at phantom-stock nodes.
- **What You Will Build #1 (GS1 & Margin-Floor Pricing Engine):** Refactor `src/services/promo_pricing.py` to validate GS1 `gtin_14` Modulo-10, use `Decimal(ROUND_HALF_UP)`, compound stackable promos multiplicatively, and clamp net price to the `margin_floor_pct`.
- **What You Will Build #2 (Real-Time True ATP Engine):** Refactor `src/services/inventory_atp.py` (`calculate_true_atp` / `evaluate_location_atp`) to subtract `reserved_qty + damaged_hold_qty + bopis_lock_qty + safety_stock_qty + unprocessed_pos_events`.
- **Deliverable & Verification:** Pass all 6 unit/integration tests in `tests/test_lab01_triage.py` (`./labs/lab-01-margin-inventory-triage/verify.sh`) and inspect `http://localhost:8002`.

tryo•labs

# Lab 1 Instructions & Copy-Paste Antigravity Prompts

Follow Steps 1–5 on the left and copy-paste prompts from labs/lab-01-margin-inventory-triage/PROMPTS.md

## Step-by-Step Lab 1 Instructions

- **Step 1 (Initialize Lab 01):** Run `./labs/lab-01-margin-inventory-triage/install.sh` to seed `data/retail_pulse.db` and starter bugs.
- **Step 2 (Reproduce Failing Tests):** Run `./.venv/bin/pytest tests/test_lab01_triage.py` to observe the margin bleed and phantom ATP failures.
- **Step 3 (Fix `src/services/promo_pricing.py`):** Validate GS1 `gtin_14` Modulo-10, use `Decimal(ROUND_HALF_UP)`, never stack `EXCLUSIVE` promos, apply stackable promos multiplicatively, and clamp price to `margin_floor_pct`.
- **Step 4 (Fix `src/services/inventory_atp.py`):** Subtract `reserved_qty + damaged_hold_qty + bopis_lock_qty + safety_stock_qty + unprocessed_pos_events` from `on_hand_qty` and clamp `true_atp = max(0, raw_atp)`.
- **Step 5 (Verify):** Run `./labs/lab-01-margin-inventory-triage/verify.sh` (`6 passed`: `$18,420.50` bleed stopped across 28 orders; 35 VIP orders flagged).

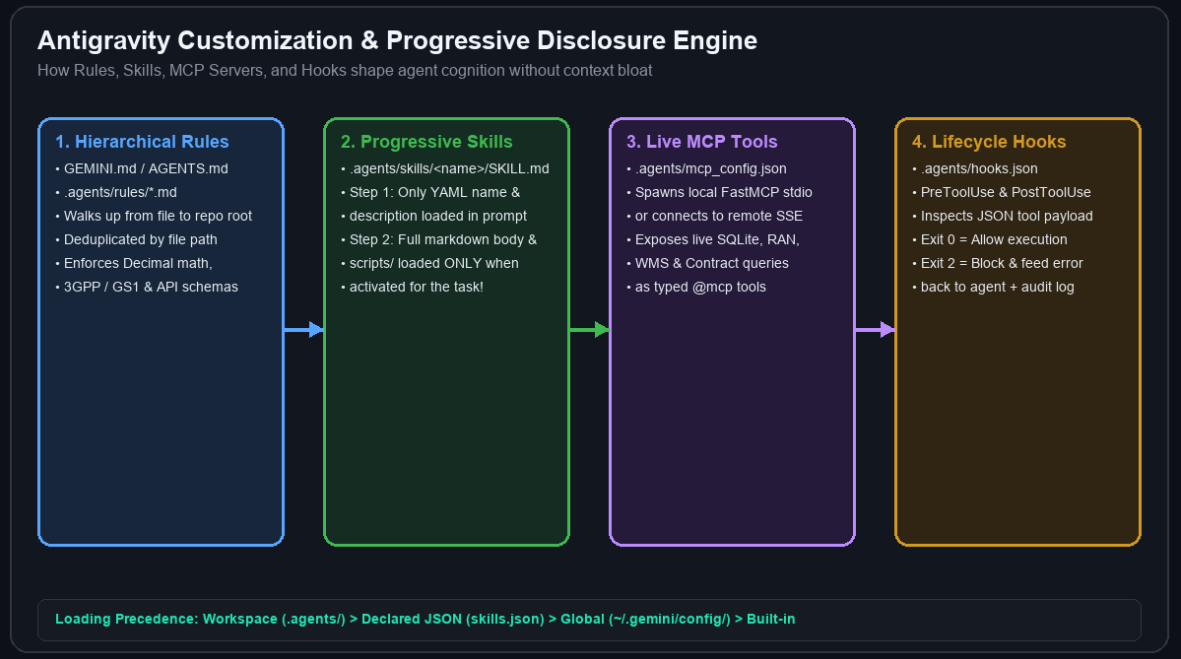
## Copy-Paste Prompts (PROMPTS.md) & Verification

- **Prompt 1 — Discovery (@Mentions):** '@src/services/promo_pricing.py @src/services/inventory_atp.py @data/logs/pos_wms_webhooks.jsonl @tests/test_lab01_triage.py Run pytest and explain why 28 orders are bleeding margin and 35 VIP orders have phantom ATP.'
- **Prompt 2 — Planning (/plan):** '/plan Create a remediation plan for promo_pricing.py (GS1 GTIN-14, Decimal, multiplicative stackable promos, margin floor clamping) and inventory_atp.py (subtract reserved, damaged_hold, bopis_lock, safety_stock, and unprocessed_pos_events).'
- **Prompt 3 — Inline Edit (Cmd+I):** Highlight `calculate_true_atp` in `inventory_atp.py` -> 'Subtract reserved_qty, damaged_hold_qty, bopis_lock_qty, safety_stock_qty, and unprocessed_pos_events from on_hand_qty.'
- **Prompt 4 — Autonomous Verify:** 'Apply fixes to @src/services/promo_pricing.py and @src/services/inventory_atp.py and run ./labs/lab-01-margin-inventory-triage/verify.sh.'

tryo•labs

# How Rules & Progressive Disclosure Skills Work Internally

Specializing Antigravity for your industry without overflowing the context window



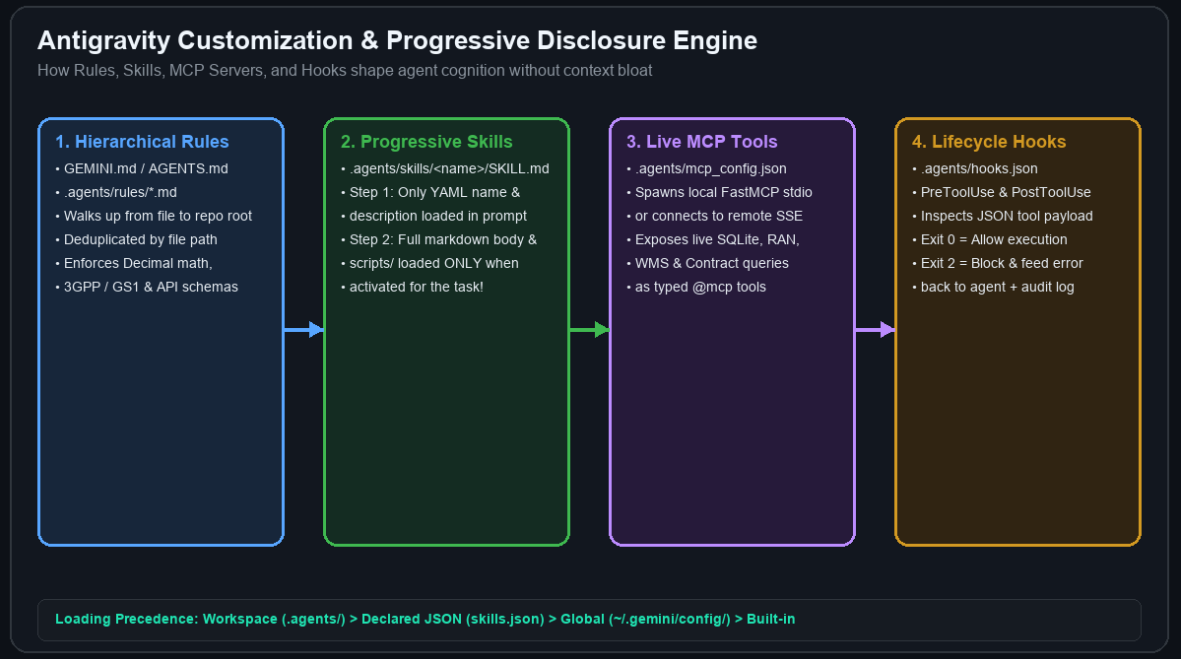

## Rules vs. Skills vs. /learn

- **Hierarchical Rules (GEMINI.md & .agents/rules/*.md):** Always-on or model-decided domain invariants loaded from workspace root down to target directories (deduplicated automatically)
- **Progressive Disclosure Skills (.agents/skills/<name>/SKILL.md):** Only lightweight YAML frontmatter (name, description) sits in system prompt; full markdown runbook & scripts/ load ONLY when triggered!
- **Self-Improving Memory (/learn):** Ask Antigravity to codify a newly discovered debugging pattern or architectural standard into a reusable workspace customization
- **Precedence Hierarchy:** Workspace (.agents/) > Declared JSON (skills.json) > Global (~/.gemini/config/) > Built-in

**Rule of Thumb: Put permanent guardrails in GEMINI.md/rules; put multi-step workflows & scripts in SKILL.md.**

# Lab 2: Workspace Rules & Progressive Skills — Objectives & Build Spec

54 Minutes • Printable PDF Guide: labs/lab-02-rules-and-skills/LAB_02_INSTRUCTIONS_EN.pdf




## Lab 2 Objectives & What You Will Build

- **Objective 1 (Hierarchical Domain Rules):** Encode permanent GS1 GTIN-14, True ATP, multiplicative promo, and Supplier OTIF invariants into workspace rules.
- **Objective 2 (Progressive Disclosure Skills):** Build an on-demand executive merchandising & supply-chain audit skill in `.agents/skills/` using lightweight YAML frontmatter.
- **What You Will Build #1 (`GEMINI.md` & `.agents/rules/retail-standards.md`):** Workspace context & rule files enforcing `Decimal(ROUND_HALF_UP)`, GS1 Modulo-10 check digits, `margin_floor_pct`, and OTIF chargeback formulas.
- **What You Will Build #2 (`.agents/skills/retail-otif-margin-auditor/SKILL.md`):** Progressive skill (`name: retail-otif-margin-auditor`) orchestrating `scripts/audit_vendor_otif.py`.
- **Deliverable & Verification:** Generate `data/exports/otif_chargeback_audit_report.json` (`$98,250.00` OTIF chargebacks across 6 suppliers) and pass all 4 Lab 02 tests (`10 passed` cumulative) in `./labs/lab-02-rules-and-skills/verify.sh`.

tryo•labs

# Lab 2 Instructions & Copy-Paste Antigravity Prompts

Follow Steps 1–5 on the left and copy-paste prompts from labs/lab-02-rules-and-skills/PROMPTS.md

## Step-by-Step Lab 2 Instructions

- **Step 1 (Initialize Lab 02):** Run `./labs/lab-02-rules-and-skills/install.sh` (auto-applies Lab 01 solution).
- **Step 2 (Author Root `GEMINI.md`):** Document `sku_catalog`, `wms_inventory`, `vendor_contracts`, `promo_campaigns`, `omnichannel_orders`, and mandatory `Decimal(ROUND_HALF_UP)` & `gtin_14` rules.
- **Step 3 (Author `.agents/rules/retail-standards.md`):** Codify GS1 GTIN-14 Modulo-10, True ATP (`on_hand - reserved - damaged_hold - bopis_lock - safety_stock - unprocessed_pos_events`), multiplicative promos, margin floor, and Vendor OTIF rules.
- **Step 4 (Author `.agents/skills/retail-otif-margin-auditor/SKILL.md`):** Add YAML frontmatter (`name: retail-otif-margin-auditor`, `description: ...`) and instructions to run `./.venv/bin/python scripts/audit_vendor_otif.py`.
- **Step 5 (Verify):** Run `./labs/lab-02-rules-and-skills/verify.sh` (`10 passed` cumulative + `data/exports/otif_chargeback_audit_report.json` generated).

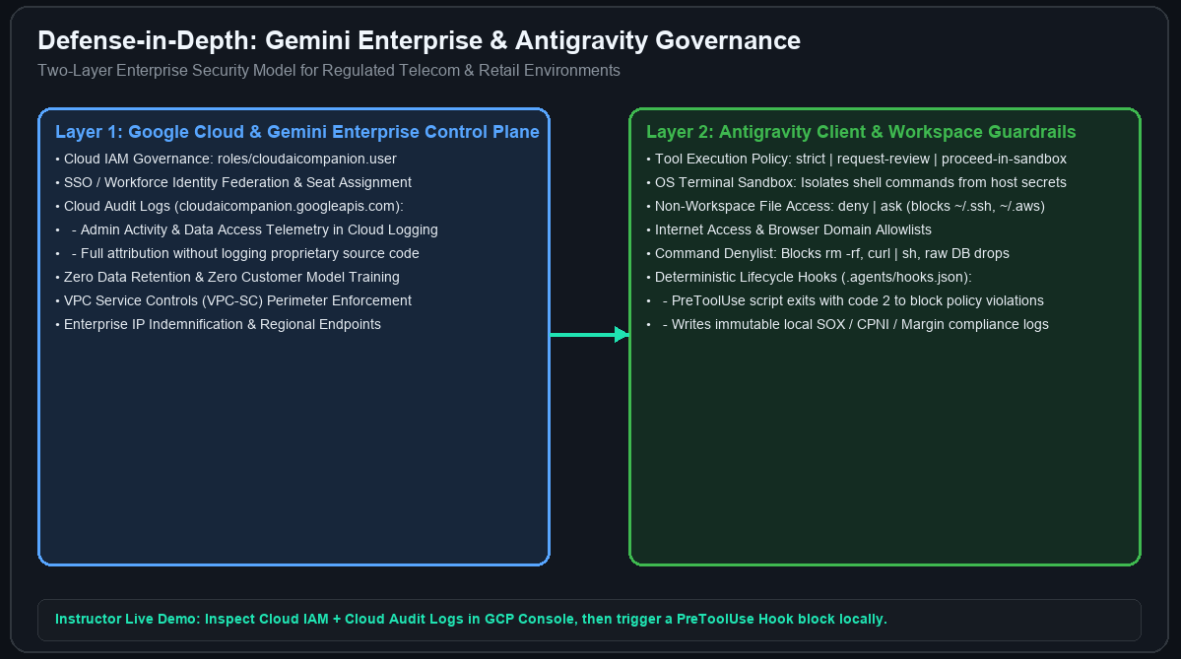
## Copy-Paste Prompts (PROMPTS.md) & Verification

- **Prompt 1 — Author Workspace Rules:** '/plan Create @GEMINI.md and @.agents/rules/retail-standards.md enforcing Decimal ROUND_HALF_UP, GS1 GTIN-14 Modulo-10 validation, True ATP deductions, multiplicative promo stacking, and OTIF chargeback rules.'
- **Prompt 2 — Build Progressive Skill:** 'Create @.agents/skills/retail-otif-margin-auditor/SKILL.md with YAML frontmatter (name: retail-otif-margin-auditor) that executes ./.venv/bin/python scripts/audit_vendor_otif.py and writes data/exports/otif_chargeback_audit_report.json.'
- **Prompt 3 — Trigger Skill On-Demand:** 'Run a full executive margin, True ATP, and supplier OTIF chargeback audit using our workspace skill.'
- **Verify Command:** Run `./labs/lab-02-rules-and-skills/verify.sh` and test `/learn`!

tryo•labs

# Live MCP Bridge, Deterministic Hooks & Gemini Enterprise Governance

Connecting live databases safely while proving end-to-end enterprise security in Google Cloud



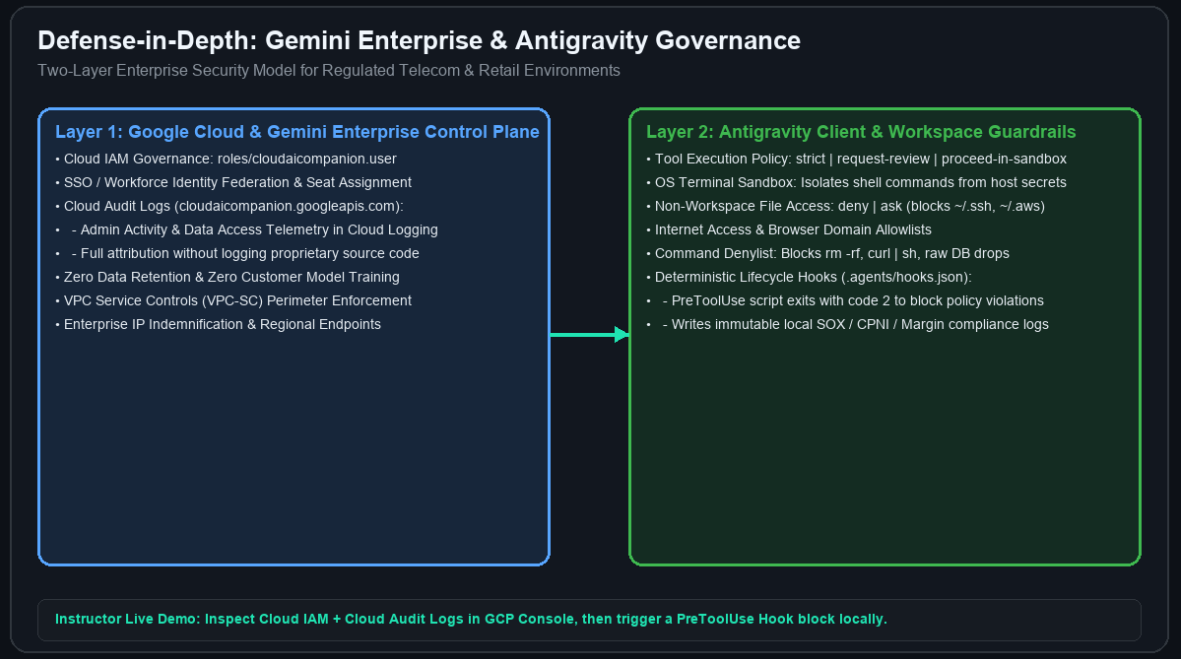

## 2-Layer Enterprise Security & Governance

- **Layer 1 — Google Cloud & Gemini Enterprise (Instructor Live Demo):** Cloud IAM (roles/cloudaicompanion.user), Cloud Audit Logs (cloudaicompanion.googleapis.com), VPC-SC perimeters, Zero Data Retention / No Training on customer code
- **Layer 2 — Antigravity Client Security:** Tool Execution Policy (strict / request-review / proceed-in-sandbox), OS Terminal Sandbox, Non-Workspace File Access (deny), Command & Browser Allow/Denylists
- **Live Context via FastMCP (.agents/mcp_config.json):** Exposes strongly-typed Python tools querying live SQLite & telemetry over stdio/SSE
- **Deterministic Lifecycle Hooks (.agents/hooks.json):** PreToolUse script inspects tool JSON on stdin; exit code 2 hard-blocks policy violations and writes an immutable audit log!

**Instructor Demo: Show Cloud Audit Logs in GCP Console + trigger a deterministic PreToolUse Hook block locally!** | **Slide 10/15**

# Lab 3: Retail FastMCP Server & Margin Hooks — Objectives & Build Spec

55 Minutes • Printable PDF Guide: labs/lab-03-mcp-and-hooks/LAB_03_INSTRUCTIONS_EN.pdf




## Lab 3 Objectives & What You Will Build

- **Objective 1 (Live Omnichannel Grounding via MCP):** Connect Antigravity to live SQLite WMS inventory, VIP orders, and vendor OTIF tables via a local `FastMCP` server.
- **Objective 2 (Deterministic Margin Compliance Guardrails):** Intercept every file-edit and shell tool call with `PreToolUse` & `PostToolUse` hooks that hard-block disabling `margin_floor_pct` or destructive SQL (`exit code 2`).
- What You Will Build #1 (`mcp_server/retail_mcp.py` & `.agents/mcp_config.json`): 4 `@mcp.tool()` functions (`query_true_atp_inventory`, `audit_vendor_otif_contracts`, `simulate_promo_margin_guard`, `list_at_risk_vip_orders`).
- **What You Will Build #2 (`.agents/hooks.json` & `scripts/margin_guard_hook.py`):** Deterministic `PreToolUse` & `PostToolUse` guardrail blocking `margin_floor_pct = 0.0`, `DISABLE_MARGIN_FLOOR`, or `DROP`/`DELETE` SQL and logging to `data/logs/hook_compliance_audit.jsonl`.
- **Deliverable & Verification:** Pass all 3 Lab 03 MCP & Hook tests (`13 passed` cumulative) in `./labs/lab-03-mcp-and-hooks/verify.sh`.

tryo•labs

# Lab 3 Instructions & Copy-Paste Antigravity Prompts

Follow Steps 1–5 on the left and copy-paste prompts from labs/lab-03-mcp-and-hooks/PROMPTS.md

## Step-by-Step Lab 3 Instructions

- **Step 1 (Initialize Lab 03):** Run `./labs/lab-03-mcp-and-hooks/install.sh` (auto-applies Labs 01–02).
- **Step 2 (Complete `mcp_server/retail_mcp.py`):** Implement `audit_vendor_otif_contracts(only_breaching=True)`, `simulate_promo_margin_guard(sku, qty=1, promo_codes_csv="")`, and `list_at_risk_vip_orders(loyalty_tier="DIAMOND")`.
- **Step 3 (Register `.agents/mcp_config.json`):** Register `retail-pulse-mcp` invoking `./.venv/bin/python mcp_server/retail_mcp.py` over stdio.
- **Step 4 (Configure `.agents/hooks.json`):** Wire `PreToolUse` and `PostToolUse` to `./.venv/bin/python scripts/margin_guard_hook.py --phase pre` (and `--phase post`, blocking policy violations with exit code `2`).
- **Step 5 (Verify):** Run `./labs/lab-03-mcp-and-hooks/verify.sh` (`13 passed` cumulative across Labs 01–03 + compliance audit log verified).

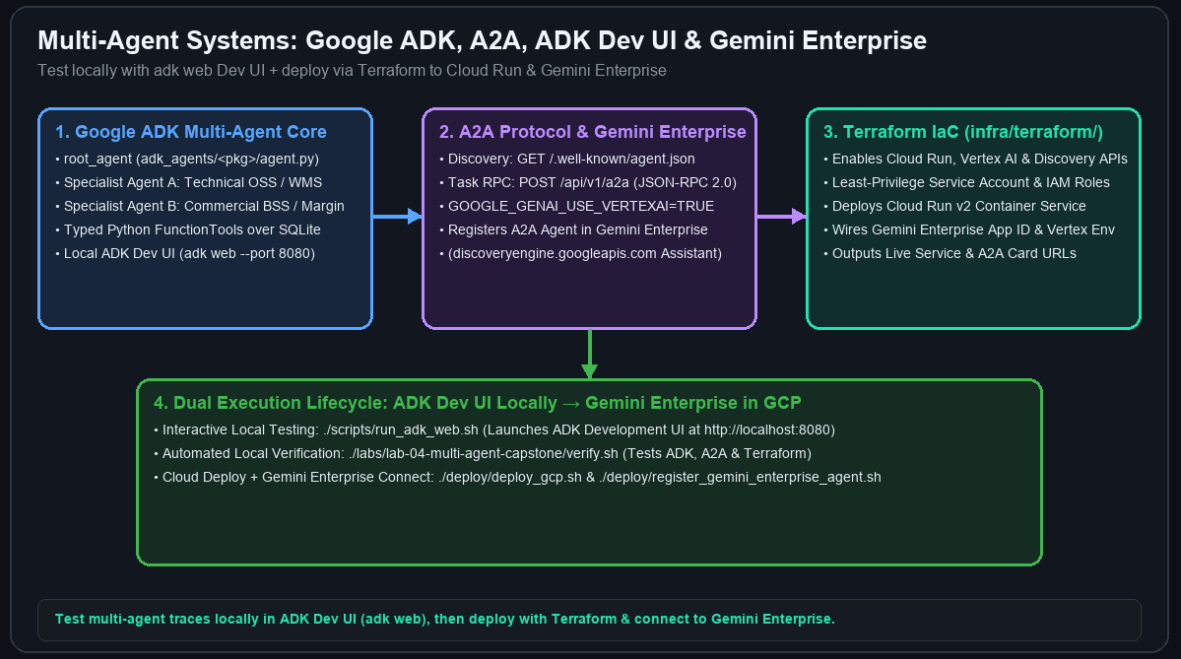
## Copy-Paste Prompts (PROMPTS.md) & Verification

- **Prompt 1 — Complete FastMCP Server:** '/plan Implement audit_vendor_otif_contracts, simulate_promo_margin_guard, and list_at_risk_vip_orders in @mcp_server/retail_mcp.py so all tool schemas match @tests/test_lab03_mcp_hooks.py.'
- **Prompt 2 — Wire MCP & Hooks Config:** 'Create @.agents/mcp_config.json for retail-pulse-mcp and @.agents/hooks.json wiring PreToolUse and PostToolUse to ./scripts/margin_guard_hook.py.'
- **Prompt 3 — Test Hook Block (Exit Code 2):** Run `echo '{"tool_name":"edit_file","tool_input":{"content":"margin_floor_pct = 0.0"}}' | ./.venv/bin/python scripts/margin_guard_hook.py --phase pre`
- **Verify Command:** Run `./labs/lab-03-mcp-and-hooks/verify.sh` and inspect `data/logs/hook_compliance_audit.jsonl`.

tryo•labs

# Multi-Agent Systems with Google ADK, A2A Protocol & Terraform

Designing multi-agent architectures locally with `adk web` and deploying to Cloud Run & Gemini Enterprise




## End-to-End Multi-Agent Architecture

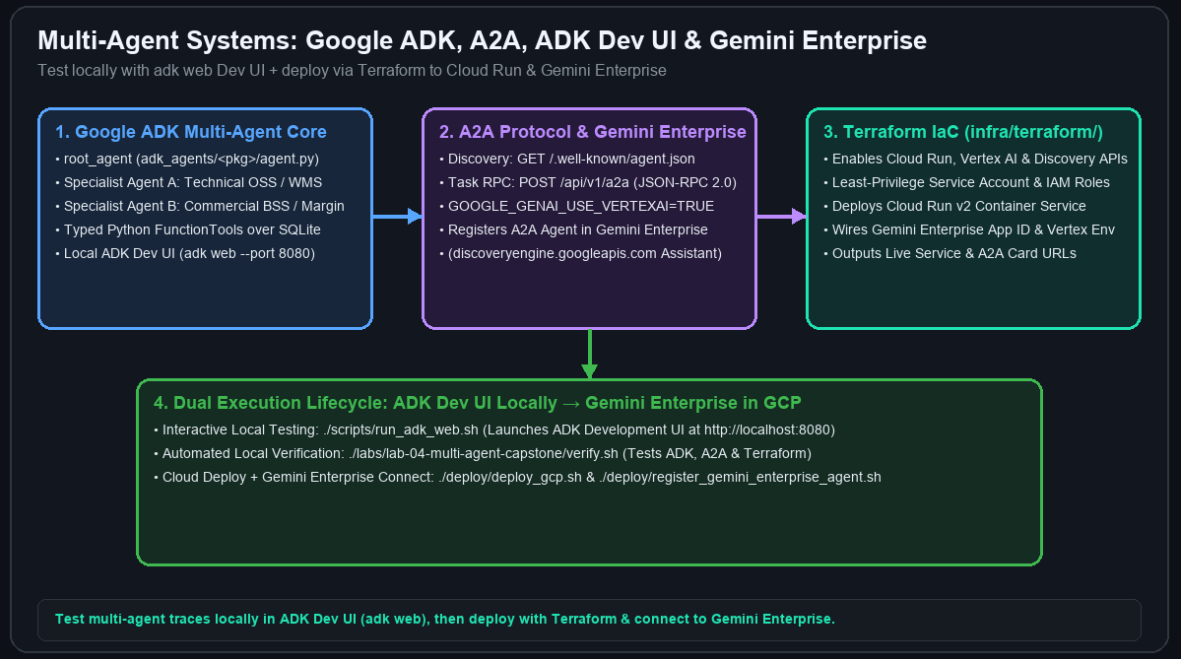
- **Antigravity Subagents (/teamwork, /boost, /grill-me):** Delegate parallel coding & research tasks inside Antigravity to build your application faster
- **Google ADK (Agent Development Kit):** Build domain multi-agent hierarchies (`root_agent` orchestrating Technical & Commercial specialist `LlmAgent`s with typed Python tools)
- **Local Testing via ADK Dev UI (`adk web`):** Launch `./scripts/run_adk_web.sh` (http://localhost:8080) to interactively inspect multi-agent event traces, tool calls, and session state!
- **A2A (Agent2Agent) Protocol:** Publish `GET /.well-known/agent.json` (Agent Card) and `POST /api/v1/a2a` (JSON-RPC 2.0 `tasks/send`) for cross-agent interoperability
- **Terraform + Gemini Enterprise:** Provision Cloud Run v2 & IAM via `infra/terraform/` (`deploy_gcp.sh`) and register your A2A agent into Gemini Enterprise (`register_gemini_enterprise_agent.sh`)

**Develop & test 100% locally with `adk web` + `pytest`, then deploy to Google Cloud & Gemini Enterprise with Terraform!** | **Slide 13/15**

tryo•labs

# Lab 4 Capstone: Multi-Agent ADK, A2A & Terraform — Objectives & Build Spec

55 Minutes • Printable PDF Guide: labs/lab-04-multi-agent-capstone/LAB_04_INSTRUCTIONS_EN.pdf




## Lab 4 Objectives & What You Will Build

- **Objective 1 (Multi-Agent System Design with Google ADK & A2A):** Build a hierarchical multi-agent system with Google ADK (`LlmAgent`) and expose an A2A Agent Card + JSON-RPC 2.0 endpoint.
- **Objective 2 (Local ADK Dev UI & Cloud Terraform Deployment):** Test agent handoffs locally in `adk web` (`http://localhost:8080`) and validate Cloud Run + Gemini Enterprise provisioning with Terraform.
- **What You Will Build #1 (`src/services/fulfillment_router.py`):** Margin-preserving omnichannel split-shipment optimizer (`optimize_at_risk_order_routing` & `claim_vendor_otif_chargebacks`) rescuing all 35 VIP orders (`$749,300.00` LTV) and claiming `$98,250.00` OTIF.
- What You Will Build #2 (`src/agents/adk_a2a_orchestrator.py` & `adk_agents/retail_pulse/`): `WMSInventoryATPAgent` + `MerchandisingMarginOTIFAgent` coordinated by `RetailPulseOrchestratorAgent`, plus `GET /.well-known/agent.json` & `POST /api/v1/a2a`.
- **What You Will Build #3 (Cloud Infrastructure):** Validate `infra/terraform/main.tf` and Gemini Enterprise registration via `./labs/lab-04-multi-agent-capstone/deploy_gcp.sh --dry-run`.

tryo•labs

# Lab 4 Instructions, `adk web` Testing & Terraform Cloud Deploy

Follow Steps 1–5 on the left and copy-paste prompts from labs/lab-04-multi-agent-capstone/PROMPTS.md

## Step-by-Step Lab 4 Instructions

- **Step 1 (Initialize Lab 04):** Run `./labs/lab-04-multi-agent-capstone/install.sh`.
- **Step 2 (Omnichannel Fulfillment Router):** Implement `optimize_at_risk_order_routing()` and `claim_vendor_otif_chargebacks()` in `src/services/fulfillment_router.py` (rerouting all 35 VIP orders to `REROUTED_SAVED` and claiming `$98,250.00`).
- **Step 3 (Google ADK & A2A Orchestrator):** Complete `src/agents/adk_a2a_orchestrator.py` (`WMSInventoryATPAgent`, `MerchandisingMarginOTIFAgent`, `RetailPulseOrchestratorAgent`, `get_a2a_agent_card`, `handle_a2a_jsonrpc`).
- **Step 4 (Local ADK Dev UI):** Run `./scripts/run_adk_web.sh` and test `retail_pulse` interactively at `http://localhost:8080`.
- **Step 5 (Terraform & Full Verification):** Run `./labs/lab-04-multi-agent-capstone/deploy_gcp.sh --dry-run` and `./labs/lab-04-multi-agent-capstone/verify.sh` (**20/20 tests pass!**).

## Copy-Paste Prompts, `adk web` & Cloud Commands

- **Prompt 1 — Fulfillment Router:** '/plan Implement optimize_at_risk_order_routing and claim_vendor_otif_chargebacks in @src/services/fulfillment_router.py ranking candidate nodes by lowest shipping_cost_per_unit and highest true_atp.'
- **Prompt 2 — ADK + A2A Orchestrator:** '/teamwork Implement WMSInventoryATPAgent, MerchandisingMarginOTIFAgent, RetailPulseOrchestratorAgent, get_a2a_agent_card, and handle_a2a_jsonrpc in @src/agents/adk_a2a_orchestrator.py.'
- **Prompt 3 — Interactive `adk web` (http://localhost:8080):** 'Execute omnichannel remediation: enforce margin floor across 28 orders, reroute all 35 VIP orders, and claim OTIF vendor chargebacks.'
- **Cloud Deploy & Verify:** Run `./labs/lab-04-multi-agent-capstone/deploy_gcp.sh --dry-run` and `./labs/lab-04-multi-agent-capstone/verify.sh`.